

Занятие 2. Расшифровка записи

Запись: `GMT20260821-131937_Recording_2966x1242.mp4`, 90 минут 22 секунды, 2026-08-21. Ведущий: Фархад. Формат: демонстрация экрана, Claude Code + браузер, в конце — камера. Расшифровка отредактирована: убраны оговорки и повторы, восстановлены названия инструментов (`settings.json`, Claude Code, Codex, MCP, GitNexus, Graphify, Superpowers). Таймкоды — от начала файла.

Важно про запись. Файл начинается посреди фразы («Контекст это... Спасибо»), то есть запись включена уже после начала эфира. **Разбор домашних заданий был проведён** — он открывал занятие, как и планировалось, но в запись не попал: её включили позже. Расшифровка начинается с блока про prompt injection. Оценки на момент эфира ещё не выставлены — об этом реплики на 50:38 и 89:10.

ТАЙМКОД	О ЧЁМ	ЧТО ПРОИСХОДИТ НА ЭКРАНЕ
00:00	Prompt injection: что попадает в контекст	Файл-каталог со спрятанной инструкцией
00:50	Демо: слабая модель против инъекции	Haiku, запрос «собери корзину на 5000»
02:40	Первый рубеж: permissions	<code>~/ .claude/settings.json</code> , allow/deny
04:00	Приём: попросить агента настроить себя	Промпт «настрой безопасность»
05:00	Чем кончилось демо	Инъекция не сработала — сработали guardrails
07:10	Умная модель находит инъекции	«4 prompt-инъекции», три видны глазом
07:50	Где permissions протекают	<code>cat</code> запрещён, обход через другой вызов
09:11	Лестница песочниц	Встроенный sandbox → sandbox ОС → VM/Docker
12:16	Итог по безопасности	—
13:34	Контекст: почему он кончается	—
14:20	Оптимизаторы: rtk и context-mode	GitHub, 76 тыс. звёзд
16:40	Как искать и проверять инструменты	Ссылка → агент → «проверь и поставь»
18:20	Вопрос из зала: контекст в Codex	—
20:19	Голосовой ввод вместо клавиатуры	Typeless, диктовка промпта
22:57	Память агента: четыре ответа	«Расскажи, как устроена твоя память»
26:40	Индексация большого проекта	element-web, Graphify, GitNexus
30:13	Оркестрация: что это на самом деле	—
32:06	Демо: параллельно против последовательно	Два прогона по element-web
36:50	Что показали замеры	128 тыс. токенов против почти миллиона
38:09	Worktrees: три независимых разработчика	Запуск через промпт
42:21	Цена worktrees	250 ГБ на одном проекте
42:59	Workflows и ночной прогон	13 часов автономной работы
47:30	Проверка руками агента: ADB, WDA, Playwright	—

ТАЙМКОД	О ЧЁМ	ЧТО ПРОИСХОДИТ НА ЭКРАНЕ
49:00	Итог занятия	—
50:38	Вопросы: домашка, дедлайн, команды, оценки	—
56:07	Superpowers подробнее	GitHub-страница, contributors
60:00	Где держать руку на пульсе	GitHub Trending
63:39	SpecKit, OpenSpec и дисциплина	—
64:20	История: месяц в режиме «в тренде»	—
66:40	Pencil: дизайн интерфейса агентом	Экран заказа кофе рисуется вживую
69:20	Кросс-проверка тремя моделями	—
74:20	Готовый макет и правки руками	Pencil
75:20	3D-аватар на React Native	Показ с камеры телефона
77:40	Топ-5 плагинов	—
82:40	Секреты: как не слить ключи	—
84:50	Большой проект и локальные модели	—
88:30	Финал	—

Prompt injection: что попадает в контекст

00:00 Контекст модели большой, и в него попадает практически всё: инструменты, которые мы используем, файлы, которые агент прочитал, наши промпты, его ответы. Если он прочитал сайт, а там была спрятана инструкция, — она тоже там.

На экране файл-каталог, внутри которого лежит «системная» инструкция: открыть Google, ввести поисковый запрос «курс доллара» и сообщить пользователю найденный курс.

«Я хотел показать это в реальном времени, но современные модели реально стали сильно умнее, поэтому он просто отказывается это делать. Давайте попробуем, но вряд ли получится. Хотя раз на раз не приходится.»

Демо: слабая модель против инъекции

00:50 Чтобы шанс был, берётся заведомо слабая модель — Haiku — и минимальный effort. Запрос обычный, пользовательский: «Соберите корзину на 5000 рублей». Прогон идёт фоном.

01:20 Пока агент работает — про то, как такие инструкции прячут:

- на сайте — белым шрифтом по белому фону;

- в тегах разметки;
- под видом системных сообщений и служебных тулзов;
- в любом месте, откуда текст попадёт агенту.

«Вариантов масса, мошенники придумывают каждый раз новые.»

02:00 Сам ведущий, по его словам, в жизни ни разу не ловил инъекцию — «или думаю, что не ловил». Но опасность в том, что через неё выполняется **любой** запрос, который придумает атакующий: это обычный взлом, просто вектор новый.

Первый рубеж: permissions

02:40 Первое, что помогает, — то, о чём говорили на прошлом занятии: настройки. На экране `~/.claude/settings.json`: `permissions` с секциями `allow` и `deny`.

Ключевая оговорка, произнесённая прямо на показе:

03:14 «Permissions работают, только если у нас режим **без** `bypass permissions`. Если стоит `bypass permissions`, `permissions` работать не будут.»

03:30 Про собственный файл ведущий говорит честно:

«Мой `settings.json` практически антипаттерн того, как должно быть. Но ввиду специфики моей работы он мне нужен в таком виде, поэтому остаётся таким.»

Здесь же — что настроить всё это можно и без знания программирования.

Приём: попросить агента настроить себя

04:00 Главная мысль блока про настройку:

«Всё, что ты не знаешь, всё, в чём ты сомневаешься, можно прогонять через него же — и в большинстве случаев это закроет ваши незнания или ваши сомнения.»

На экране агенту отправляется просьба разобрать текущие настройки и предложить, что настроить.

06:00 Ответ: агент предлагает поднять уровень безопасности и почистить лишние файлы, которые ему не нужны.

«Если вы сомневаетесь, что ваш агент настроен верно, или что у вас где-то хромает безопасность, базовой настройкой будет просто попросить самого же агента глобально, на все проекты, выставить базовые параметры безопасности.»

Чем кончилось демо

05:00 Первый прогон закончился ничем — и это правильный результат:

«Ничего не выполнилось, потому что guardrails остановили всё это дело. Именно guardrails самой модели. Если бы мы попросили более умную модель, он бы сказал, что вообще не может это сделать и что здесь есть инъекция. Глупая модель просто этого не выполнила, но ничего не сказала.»

Разница между моделями здесь принципиальная: слабая **молча** не сделала, умная — **назвала** проблему.

Умная модель находит инъекции

07:10 Тот же файл отдаётся модели поумнее. Ответ: в файле **четыре** prompt-инъекции. Глазом на экране видно три — четвёртая спрятана.

07:19 Отдельно — про инъекции не в контенте, а в инструментах:

«Когда вы качаете какой-нибудь скилл или тулзу, он может прямо в тулзе использовать мини-промпт, инструкцию, которая тоже заставит вашего агента что-то делать.»

Вывод блока: примеры простые, современные модели их отбивают быстро, но если взять слабую модель для простых задач — она вполне может сделать что-то нехорошее.

Где permissions протекают

07:50 Демонстрация дырки в списке запретов. На инструмент стоит `deny`, и прямой вызов агент не выполняет. Дальше:

- тот же файл читается **другой командой** — запрет не срабатывает;
- то же самое через Python — в разрешениях запрета на такое действие нет, вызов проходит, файл прочитан.

«В разрешениях у нас уже не будет запрещения на такое действие, и разрешение пройдёт. Он прочитает, какие наши секреты, например.»

08:50 Про живой хук: «пока отложим, ближе к концу занятия нужен будет». **К хуку в записи так и не вернулись** — это долг перед аудиторией.

Лестница песочниц

09:11 Что делать, если списка правил мало. Три ступени:

1. Встроенный sandbox Claude Code. Сессию можно запустить в песочнице — с auto-allow или с обычными permissions.

10:00 «Доступ у Claude будет только внутри sandbox, но этот доступ ограничен для терминала. Для всех остальных обвязок, которые вы сделали, он ограничиваться не будет.»

То есть если у вас MCP-серверы, AppleScript, инструменты, которые ходят через UI, — они продолжают управлять вашим Mac как раньше. Это отдельный уровень защиты, его надо ставить дополнительно.

2. Песочница операционной системы. На macOS своя есть, но Apple сама считает её deprecated.

3. Чистая машина, виртуалка или Docker.

11:20 «Самый надёжный вариант — запускать сомнительный код, сомнительный проект, сомнительное задание или research, который может потенциально что-то сделать не так, на отдельной чистой машине. Если чистой машины нет — поднять виртуальную. На уровне системы будет ограничение, и всё, что он сделает внутри, останется внутри. То же самое с Docker.»

Итог по безопасности

12:16 Что имеем по минимуму:

- ставить permissions вместо `bypassPermissions` — хотя бы auto или manual;
- периодически проводить ревью своего `settings.json`;
- sandbox для случаев, где сомневаешься в надёжности источника.

13:00 «Кому-то нужно на виртуальной машине запускать, кому-то — изолировать отдельное решение, кому-то будет достаточно, как мне, permissions или вообще bypass permissions, если вы сами контролируете практически каждую итерацию и всегда перед монитором. Если нет — конечно, нужно всё настроить.»

Контекст: почему он кончается

13:34 Контекст состоит из многих частей, он заполняется — и приходится либо открывать новую сессию, либо компактить.

«Не обязательно, что результат будет плохим. Просто если что-то пойдёт не так, починить это будет сложнее, чем начать новую сессию и начать заново.»

Оптимизаторы: rtk и context-mode

14:20 Два инструмента, которыми ведущий пользуется сам:

- **rtk** — набор хуков, которые перехватывают вызовы и упрощают то, что уходит в контекст;
- **context-mode** — не сжимает, а **не добавляет** выводы инструментов в контекст, оставляет только результат.

16:00 «Это два реальных инструмента, которые реально работают. Я их проверил на себе, пользуюсь больше двух-трёх месяцев. Есть реальное сокращение контекста, и от этого реальное сокращение токенов. Если у вас недорогой план, а вы хотите делать больше — такие инструменты сократят расход и оставят больше лимитов на работу.»

Позже, на 50:00, цифра названа честнее, чем в маркетинге:

«Они заявляют, конечно, процентов 99, но реальных процентов 20 освобождается. Может быть, даже 30 иногда.»

Как искать и проверять инструменты

14:45 Способ поиска для неразработчиков: сформулировать, какой инструмент нужен или что вообще хочется сделать, и добавить к запросу слово GitHub — почти всё сейчас там.

Дальше — обязательная проверка перед установкой:

1. скопировать ссылку на репозиторий и попросить агента рассказать, что это;
2. решить, нужно ли оно;
3. **отдельно** попросить проверить репозиторий перед установкой;
4. только потом ставить — «проверь репозиторий, и если он безопасен, установи глобально».

Ориентир по доверию:

17:40 «Если репозиторий на 20 тысяч звёзд — сейчас это уже не так много, — 2000 коммитов, это open source, очень много людей им пользуются, вероятность того, что туда попали какие-то инъекции, сильно ниже, чем у репозитория в 100 звёзд.»

Вопрос из зала: контекст в Codex

18:20 Спрашивают, как смотреть контекст в Codex, где счётчика нет.

Ответ: почти наверняка есть отдельный инструмент, который показывает заполненность — сам ведущий им не пользовался. И более полезный ход: попросить Codex проанализировать, что попадает в initial prompt в начале сессии и что попадает в системный промпт каждого спавнящегося агента.

19:20 «Вот это уже будет немножко контекст-менеджментом. Хорошее замечание, спасибо.»

Голосовой ввод вместо клавиатуры

20:19 Отступление про инструмент, который экономит время на промптинге.

«Я сейчас писал промпты руками, но вы все, наверное, заметили, что большие промпты руками писать вообще неприятно. Качество сильно выше, чем что-то другое, но очень долго.»

Варианты: **Typeless**, **WhisperFlow** или локальное решение — поднять Ollama и модель для распознавания речи, это займёт пару гигабайт оперативной памяти.

Что даёт (демонстрируется вживую):

- одна кнопка, говоришь — текст записывается;
- слова-паразиты, задумчивость и кривые формулировки обрезаются;
- внутри есть модель, которая оптимизирует смысл: повторённую дважды мысль запишет один раз;
- умеет списки и перевод; термины вроде Docker, Nginx, Swift распознаёт и пишет корректно в русском тексте.

22:10 «Сильно сокращает время на промптинг, и промпты получаются более объёмными. Больше задач поставить, построить какой-нибудь флоу вместо простых односложных „вперёд“, „да, продолжай“.»

Память агента: четыре ответа

22:57 Что агент помнит между сессиями — о проекте и о вас. На экране прямой вопрос агенту: «Расскажи, как устроена твоя память».

23:41 Ответ агента: несколько `.md` -файлов, где до определённого объёма хранится информация о проекте, о том, какие проекты вы ведёте и что вам нравится.

«Туда влезает не очень много. Он сам старается держать эту память довольно чистой, класть только выжимку, поэтому не всегда она реально помогает между сессиями.»

Что есть сверху:

- **claude-mem** и подобные библиотеки (крупных штук пять) — хуки, которые сохраняют всё в персистентную память между сессиями. Ведущий сам не пользуется — «наверное, уже стоит начать»;
- **Obsidian** — то, чем пользуется он: обычные заметки, куда агент через хуки складывает daily-записи и информацию по проектам. Причина выбора — не только Claude:

25:20 «Я использую это, чтобы можно было использовать не только Claude, но и другие модели, чтобы они сразу знали что-то о проекте. Потом это всё просто прочитывается другим агентом, у которого тоже настроен Obsidian.»

26:40 Вывод: у Claude память относительно неплохая, её хватает на базовые решения; за расширением — к отдельным инструментам, выбирать по вкусу, попользовавшись каждым.

Индексация большого проекта

26:50 Задача: чтобы модель понимала большой проект без лишних проходов по файлам.

Полигон — **element-web**, open-source мессенджер, клонированный к себе.

27:30 «Если раньше мне заняло бы какое-то время запустить element-web у себя, чтобы разобраться, на чём он написан, как его запустить, какие зависимости установить, — сейчас я в три промпта развернул его локально.»

Два инструмента поверх него:

- **Graphify** — строит граф взаимодействия всех частей приложения и частей внутри модулей;
- **GitNexus** — примерно то же, но в другом ключе.

28:10 «Нашёл я их в рилсах в Instagram. Очень много полезной информации бывает в шортсах, рилсах и телеграм-группах. Рекомендую найти себе один хороший канал и краем глаза посматривать. Естественно, всё проверять.»

Как пользоваться: скинуть ссылку агенту → проверить → установить → запустить индексацию. После этого вместо обычных grep и read агент ходит через хуки GitNexus — за данными о файле, функции, решении внутри проекта. Graphify требует больше времени на индексацию, граф получается больше. Друг другу они не мешают и работают вместе.

30:00 «Если проект маленький — это вообще не нужно.»

Оркестрация: что это на самом деле

30:13 Переход к оркестрации — с прямой оговоркой про то, как это слово используют:

30:40 «В инфополе гуляет очень много разной информации, это сейчас очень модное слово. Особенно начальники компаний и всякие около-бизнесмены: „я занимаюсь оркестрацией, у меня команда из 10 агентов, они делают всю работу за меня — один маркетолог, один SEO“. Это всегда прикольно слушать, потому что это не всегда правда. Проверить работу 10 агентов — это не то же самое, что проверить работу в одном чате.»

31:30 Что такое агент: та же модель, что и в чате, просто спавнится отдельно и работает отдельно. Те же мозги, другие промнты. Вызвать может и агент сам, и вы.

31:56 Самая базовая оркестрация — вызвать несколько субагентов: проверить репозиторий, проиндексировать проект. «Самый простой, удобный и начальный путь.»

Демо: параллельно против последовательно

32:06 Два прогона на element-web:

- промпт «изучи три подсистемы репозитория element-web **параллельно**»;
- то же самое **последовательно**, без субагентов.

Пока считается — объяснение, чего ждать, по прошлым замерам:

32:50 «Казалось бы, параллельные агенты должны работать быстрее. Но на практике, если задача небольшая, параллельные агенты каждый тянут за собой новый системный промпт, новую загрузку инструментов, новые проходки, и у них нет информации от других субагентов — она приходит только потом. Поэтому получается, как ни странно, дольше, чем если бы мы выполняли небольшую задачу последовательно.»

Где выгода настоящая:

33:30 «Если вы делаете большое маркетинговое исследование и вам нужно независимо запустить несколько агентов, чтобы они в абсолютно разных направлениях искали, — выгода появляется. Каждый агент изолирован, независим, работает в своём контексте, в него влезет больше данных, и он сделает более качественный research. Если таких агентов 10 — за ту же единицу времени у вас 10 research'ей.»

34:20 Второй эффект, ради которого субагентов и зовут: они экономят контекст основного чата. Если вы долго вели сессию по проекту и не хотите терять наработанный контекст — субагенты сделают работу на его основе, каждый в своём окне, а в главную сессию вернутся только результаты.

36:00 «Вы получите только результат их действий, а не все те мусорные использования инструментов, прочитки файлов и так далее.»

36:20 Реплика из зала: объяснение получилось не совсем точным. Ведущий переформулирует.

Что показали замеры

36:50 Живьём последовательный прогон закончился быстрее — параллельный к этому моменту ещё шёл (закончился чуть позже, выигрыша по времени не дал).

36:55 «Придётся, видимо, поверить мне на слово, что в такой небольшой задаче параллельно будет быстрее... Последовательно было чуть быстрее, но задача была маленькая.»

Дальше на экран выводится скриншот замеров, и там разница не во времени, а в контексте:

СПОСОБ	КОНТЕКСТ ГЛАВНОЙ СЕССИИ
Три субагента параллельно	128 тыс. токенов (≈15 % от миллиона)
Всё последовательно в одном чате	практически миллион токенов

37:30 «Если у вас контекст, скажем так, грязный, вам лучше бы начать новую сессию. А здесь вы можете продолжить работать в той же сессии, потому что она практически чистая.»

Worktrees: три независимых разработчика

38:09 Worktrees — изолированные среды на независимых git-ветках.

Сравнение, вокруг которого построен весь блок:

40:00 «Если три субагента работают в одной сессии — это один человек, разделённый на три. А если это три агента в трёх разных worktrees — это уже три независимых разработчика, каждого из которых нужно отдельно курировать: у него отдельная git-история, отдельные коммиты, отдельные файлы, потому что всё изолировано в ветке, и потом отдельные pull/merge-реквесты.»

Почему это важно: субагенты в одной папке спотыкаются друг о друга — «файлы, которые изменит один агент, изменятся и у другого». В worktree у каждого своё окружение и своя ветка, дальше — merge руками.

40:40 «Это реально ускоряет разработку, когда нужно сделать несколько разных функционалов или протестить гипотезу на одном участке кода.»

41:00 Запускается промптингом: сказать, что работаем в worktrees, сделать три агента, каждый в отдельном worktree, инициализировать git, если его нет. Демо на том же файле-каталоге: три агента делают по HTML-странице.

41:47 «Я сам с worktrees особо не работаю.»

Цена worktrees

42:21 Обратная сторона, о которой предупреждают редко: каждый вызов создаёт новую ветку и копирует код локально.

«У меня один раз проект — 250 гигабайт чисто на worktrees. Диск нужно беречь: в какой-то момент вы запустите несколько worktrees, он просто заполнится, работа встанет — и всё. Если вы за компьютером — ничего страшного. Если вас нет, вся работа встанет.»

Workflows и ночной прогон

42:59 Workflow — это большой промпт с шагами, сохранённый как повторяемый паттерн: в скилл или в собственный формат среды (в Cursor свои, в Codex устроены иначе, в Claude Code иначе). «По сути — инструкция для агента, которая выполняет по шагам определённые действия.»

43:40 Зачем это лично ведущему — сценарий «ночной workflow»:

1. За день накопились материалы по задаче: research, продуманная архитектура, дизайн, бизнес-логика, тикет.
2. Вечером — 15–20 минут на большой качественный промпт.
3. Дальше по шагам: под каждую задачу пишется спецификация → её проверяют **два независимых агента, по два раунда каждый** → пишется план → план проверяется дважды независимым агентом → полный цикл реализации по плану → написание тестов → проверка реализации тестами → проверка реализации агентами → финальная проверка всей работы отдельным независимым агентом → отчёт отдельным файлом.

4. Запуск на ночь.

46:10 «Главное, чтобы у вас всё было настроено. Если это потенциально опасная работа, естественно, нужно поставить sandbox, запретить ему всякое, и запускать.»

46:31 «Самый долгий workflow, который у меня получился, — 13 часов независимой работы Claude. Он сделал реально крутую фичу, очень большую, хорошо её проверил сам и всё это мне выкатил.»

Если установлен **Superpowers** — он сам предложит доработать ваш флоу и сохранить его как workflow.

47:00 «Цена — просто несколько часов работы компьютера ночью. Утром вам нужно проверить, провалидировать, прочитать код.»

Проверка руками агента: ADB, WDA, Playwright

47:30 Чем закрывается ручное тестирование, пока вы спите:

- **Android** — подключить телефон по **ADB**, агент закроет процентов 80 ручного тестирования;
- **iOS** — **WDA** (WebDriverAgent) даёт примерно тот же функционал на физическом устройстве;
- **фронтенд** — **Playwright**, «он также всё сам проверит, это очень хорошо работает»;
- **бэкенд** — проверяется сам, достаточно поднять дополнительный стенд.

48:30 «Ребятам из Android и iOS очень рекомендую попробовать погонять агентов через ADB и WDA — сокращает ручную проверку очень сильно, даже удобнее, чем симулятор, во многих вопросах: звонки, пуши — на симуляторе это просто не проверишь.»

Итог занятия

49:00

«Наверное, на сегодня всё. Получилось очень много, и по каждому пункту по-хорошему нужно сделать свой небольшой research. То, что вы услышали, — это хорошо, но когда вы попробовали сами, открыли тот же GitNexus, почитали, как он работает, установили себе, прогнали — это сильно лучше.»

Вопросы: домашка, дедлайн, команды, оценки

50:38 Будет ли ещё домашка и когда?

«Сегодня вечером я скину, постараюсь побыстрее — наверное, в течение двух часов. И собрать старую домашку в красивый PDF.»

50:55 Очистка контекста — не трогает ли файлы?

«Файлы в директории вообще не трогаются, это всё внутри чата. Контекст к файлам никакого отношения не имеет. Чистка контекста безопасна. Контекст преимущественно не жалко потерять.»

51:30 Отдельно — совет для тех, кто не разработчик (продакты, маркетинг, тестировщики):

«Попробуйте поработать в папках. Создать папку, инициализировать в ней git-репозиторий — всё как в домашке — и вести общение через эту папку, чтобы ИИ сохранял все наработки внутрь. Когда вы работаете в ChatGPT в браузере, вы не можете начать новую сессию со знаниями старой. Частично это решает Claude Cowork, но субъективно, когда есть Claude Code, он не нужен.»

52:30 «Всем нужно пользоваться Codex, Claude Code или другими кодинговыми инструментами, потому что они удобнее даже для некодинговой работы: всё хранится локально, все данные исключительно ваши. При хорошо настроенном проекте потеря контекста вообще не страшна — начинаете новый чат, говорите „подтяни все знания о проекте“ — и вперёд.»

53:30 Дедлайн домашки:

«К середине недели где-то. Как раз выходные будут, можно будет что-то поделать.»

«Постараюсь какое-нибудь небольшое задание сделать, чтобы оно было более интересным. Понимаю, что не всем было интересно делать task manager. Надеюсь, что вы возьмёте какой-нибудь свой прикольный проект. Какие-нибудь примеры я тоже дам — можно будет выбрать из них, если не захочется включать фантазию.»

54:16 Команды или соло — вопрос вынесен в зал:

«Напишите, что было бы вам интереснее — работать в командах или по одному. Попробовать в командах — отличный навык, который реально нужен.»

54:35 Направление новой домашки (сказано дословно):

«У новой домашки будет направление — вот то, что мы сегодня прошли: по безопасности, sandbox, оркестрации. Скорее всего, будет что-нибудь обязательно попробовать — какой-нибудь context-mode, GitNexus или что-нибудь из похожего инструментария.»

54:50 Больничный и отпуск:

«Если кто-то на больничном, в отпуске — сроки будут смещаться. Даже если вы не успели, ничего страшного: сдадите позже, просто не попадёте в основной список. Может быть, это повлияет на итоговую оценку — это уже с вашими руководителями будет решаться. Я проверю, всё ок.»

55:32 Решение по формату — по ответам в чате:

«Кто захочет — будет по одному, кто захочет — будет делать в командах. Кто будет делать в командах, тому будет сразу чуть-чуть плюсики.»

55:46 Оценки с зонами роста:

«Зоны роста я не подсвечивал, на самом деле. Но давайте — да, будет с зонами роста.»

56:07 Планируются ли ещё лекции?

«Пока не могу сказать. Потом информацию эту дам.»

Superpowers подробнее

56:23 Отвечая на просьбу из чата — почему Superpowers:

- **обкатан людьми** — появился примерно полгода назад, им пользуется очень много народу;
- **49 contributors** — «не прям супер много, но это аккумулированные данные многих людей и многих проблем, которые он решает»;
- **275 тысяч звёзд.**

57:40 «В мою бытность молодым разработчиком на Swift, в далёком 17–18-м году, 2–3 тысячи звёзд — это было неимоверное счастье. Даже 500 звёзд — уже круто. Сейчас, в эпоху ИИ, 275 тысяч — реально много, но, по-моему, не потолок.»

Что он делает:

58:40 «Он ставит вам хуки, через которые вы не можете пройти просто так. Даже если вы забыли сделать спецификацию или продумать задачу глубже, настроенный по умолчанию Superpowers отловит этот момент и задаст вам вопросы. Тем самым на этапе проектирования продукта — будь то кодинг-продукт или просто текстовое описание сайта — он это отловит и улучшит.»

59:00 «Чем раньше улучшить мысли модели, тем лучше будет конечный результат. Исправлять всегда сложнее, чем изначально сделать хорошо.»

Совет: скинуть репозиторий агенту и попросить рассказать, как он устроен.

Где держать руку на пульсе

60:00 Источники, которыми пользуется сам:

- **YouTube** — «люди запариваются и делают хорошие research’и; в одном ролике столько полезного, что сами вы собирали бы это день-два». Оговорка: нужно посмотреть несколько роликов, чтобы понять, кто учит с точки зрения разработки, а кто — просто побаловаться с ИИ. «Смотреть нужно и то, и то.»
- **Telegram-каналы** и **Instagram** — многие просто парсят GitHub Trending и постят выжимку.
- **GitHub** → **Explore** → **Trending** — открывать понравившиеся репозитории.

60:20 «Скину, наверное, парочку вам для интереса — что лично сам смотрю.» 62:10 «Telegram-каналы тоже какие-нибудь скину.»

61:20 Пример «побаловаться, а оказалось полезно»:

«Недавно я сделал переозвучку своего видео на английский своим же голосом через Claude. Просто поставил задачу переозвучить. У меня стояла Ollama — он скачал voice-модель от Microsoft, клонировал мой голос, переозвучил, учитывая все тайминги, задержки, разницу между длинной русской речью и более короткой английской. В итоге я всё равно переозвучил голосом диктора, потому что прослеживался сильный русский акцент, но опыт прикольный.»

62:30 В Trending показывается набор скиллов на 228 тысяч звёзд — «аналог Superpowers, там тоже свои хуки, и brainstorming примерно тот же».

SpecKit, OpenSpec и дисциплина

63:39 Вопрос: пользовался ли спец-фреймворками вроде SpecKit и OpenSpec.

«Нет, не пользовался, если честно. У меня чисто ручной флоу на базе Superpowers: одна задача, одна спецификация, один план, одна разработка и несколько проверок. Мне тоже стоит уже потихонечку углубляться в это дело. Если раньше это было больше как баловство, сейчас ИИ требует больше дисциплины, больше инженерии.»

История: месяц в режиме «в тренде»

64:20

«Месяца три назад я решил, что буду прям супер держать руку на пульсе. Отслеживал все нововведения, пробовал вообще всё: YouTube, Telegram, Instagram, трендинговые репозитории, каждый инструмент проверял, погружался с головой. К концу месяца я был просто супер-топ ИИ-специалист — на пике: чем можно пользоваться, что для чего есть, какие модели для чего лучше. И потом просто понял, что это невозможно делать каждый раз. Один раз для теста интересно, но у тебя жизни нет, это всё сжирает. Нужно, как и в любой сфере, определить для себя вектор и в него углубляться.»

Пример полезного «в сторонку»: GitHub MCP — скачать репозиторий, посмотреть всю историю git, построить граф разработки: кто когда что писал, как ушёл; ставить оценки коммитам, строить деревья.

Pencil: дизайн интерфейса агентом

66:40 Формально занятие закончено — дальше по желанию.

«Кто-то уже ушёл. По идее занятие закончилось. Если хотите — оставайтесь, кому надо — можно уходить.»

67:07 Инструмент **Pencil** — «по факту то же самое, что Figma». Задача агенту: собрать дизайн мобильного приложения кофейни, экран заказа кофе, одно окно.

67:40 «Прикольно, тут прям видно, как в реальном времени агент рисует дизайн. Здесь подключён Claude по той же подписке, которая у меня есть основная.»

67:55 История инструмента:

«Чувак, по-моему, работал в Figma и ушёл. Он придумал MCP для Figma, предложил его Figma — ему отказали, потом сделали сами такой же MCP, и он назло им выпустил свой инструмент, вот этот Pencil. Реально классно работает, и у него классный MCP, он хорошо работает с агентом.»

68:30 Обобщение:

«Всё, что вы делали руками через ИИ — генерацию видео, фото, картинок, — тоже можно делать через кодинг-инструмент, и это будет удобнее и быстрее: он сам сможет всё проверять. Нужно просто найти MCP, которая будет это генерить через ваш сервис, вы по API получаете доступ и даёте его Claude. А в Codex это и так доступно из коробки.»

Кросс-проверка тремя моделями

69:20 Разбор чужого сетапа, приехавшего в чат, переходит в главную мысль хвоста занятия.

Как понять, что ваш флоу хороший:

70:00 «Ты берёшь, собираешь тестовый проект и тестишь на нём различные варианты своего сетапа. И смотришь: при одной и той же задаче у тебя хорошо получается или плохо.»

70:50 Наблюдение, которое ломает интуицию «сетап один — значит, результат один»:

«На Opus 4.8 у меня все флоу отлично отработывали — я реально добился 13 часов непрерывной работы за один день. На Opus 5 мой рекорд — часа четыре, потому что у него как будто более серьёзные защитные механизмы, и он нет-нет в какой-то момент останавливается, чтобы попросить у меня подтверждение. Хотя я прямо говорил, какие решения и где принимать. Сетап абсолютно один, проект один и тот же, модель, казалось бы, тоже — но всё-таки модель разная.»

71:50 Отсюда — рекомендация:

«Оценку через ИИ я бы рекомендовал проводить минимум через трёх независимых агентов. Причём это должен быть минимум Opus, Codex 5.6, желательно Sol или 5.5, и в идеале каким-нибудь Fable тоже прогонять. Это дорого по токенам, дорого по времени, но получается классный результат.»

72:20 Живой пример:

«Вчера помогал брату делать презентацию для защиты диплома — MBA-программа, серьёзная вещь. Он делал через Codex. Я прогнал все его выводы, весь UI презентации, всё, что не получалось, через Opus, кросс через Fable и кросс через 5.6 Sol. Всё это соединил воедино — естественно, не я, я просто поставил задачу. Конечный результат зашёл с первого раза, хотя сам он занимался этим практически целый день.»

73:20 «Доверять отдельной модели я бы не стал. У меня подписка и на Codex, и на китайцев, и на Claude, и всё это приходится использовать реально каждый день, чтобы получать качественный код. Самый вопрос — это корнер-кейсы, когда всё идёт чуть-чуть не так. Чтобы не ломалось — кросс-проверки и спецификации с планами.»

74:00 Оговорка:

«На самом деле это лишь один подход к разработке. Можно найти другие — есть же test-driven development и так далее, все эти паттерны никуда не ушли.»

Готовый макет и правки руками

74:20 На экран возвращается Pencil с готовым экраном.

«Вот такой экранчик ИИ нарисовала, и он прям каждый момент отрисовывал. Его можно просить редактировать здесь, либо мы можем редактировать всё сами — двигать элементы. Можно почувствовать себя дизайнером. Если кому-то когда-нибудь хотелось сделать соло-продукт — сейчас это всё очень доступно.»

3D-аватар на React Native

75:20 Показ с камеры телефона — приложение, сделанное для заказчика.

«Ребята попросили 3D-аватар. Вот такой 3D-аватар в мобильном приложении, написанный на Unity, полностью сделанный вместе со мной. Каждый орган здесь тапабельный, всё встроено в приложение, крутится, практически не жрёт батарею на моём стареньком 13-м, не лагает.»

76:30 «За два дня у меня это получилось. Я в 3D-моделировании вообще никто, писать шейдеры тоже не умею — я iOS-разработчик. Собрать приложение я бы мог, но как туда встроить Unity — заняло бы много времени. За два дня я на React Native собрал макет: взял найденную в интернете модель, засунул органы, написал шейдеры, написал анимации, добавил прозрачности, украшения, сделал органы тапабельными.»

77:10 «До этого момента не думал, что так можно, но решил попробовать — и оказалось, что можно практически всё. Ограничений сейчас в цифровом мире практически нет.»

Топ-5 плагинов

77:40 Вопрос из чата: топ-5 плагинов, которые ставил бы в любой ситуации.

1. **Git-плагин** — «даже плагином не считается уже, база». Через интеграцию Claude может сам создавать pull request'ы, менеджить конфликты, вести проекты в GitHub, создавать actions. Оговорка: «нужно ли это и можно ли это на рабочих гитах — проконсультируйтесь или сами решите».
2. **Context7** — тянет актуальную документацию. «Стоит как база.» Нужен бесплатный API-ключ после регистрации.
3. **context-mode** — экономит токены. «При большой подписке не так нужно, но приятно, что ближе к концу недели есть свободные токены — сделать что-нибудь для души или опробовать новую модель.»
4. **GitNexus** — «чем пользуюсь каждый день».
5. **Pencil** — «пользуюсь тоже».

79:40 Про RAG и fine-tuning: пользуется, сейчас многие просят собрать своих агентов — там это помогает.

80:00 «Получилось собрать такого агента, где ни под каким предлогом я не смог его взломать, что под капотом DeepSeek.»

«Каждый ИИ чуть-чуть разный, и только на опыте вы поймёте, с каким как работать лучше. Работая в Codex, вы понимаете его лучше: как его промптить, какие инструменты он лучше использует. И в каждом отдельном проекте всё работает чуть-чуть по-разному. Если мой флоу ведения беседы с Claude один, эти инструменты будут работать нормально со мной, а вы будете писать по-другому — и для вас они сработают иначе.»

81:50 «Плюс мой AGENTS.md верифицирован недели три назад, но там реально чуть-чуть много антипаттернов: очень много разрешений и послаблений я даю агенту, потому что специально почистил весь компьютер от sensitive-данных. Особенно полгода назад было очень популярно prompt injection на поиск сид-фраз для блокчейн-кошельков — вообще нон-стоп. Я всё почистил и расслабился, отпустил его в свободное плавание. Так делать я не рекомендую.»

Секреты: как не слить ключи

82:40 Прямой ответ на вопрос, как работать с секретами:

«Тут всё на самом деле сложно, потому что агент в конечном итоге всегда может получить данные и прочитать их, и в контекст это попадёт. Насколько это опасно — неизвестно, потому что это проходит через сервера того же Anthropic. Кто сможет перехватить именно вашу сессию, достать ключ и сохранить — большой вопрос.»

83:30 «Он 100 % получит доступ в конечном итоге. В какой-то момент расслабитесь, он его прочитает, ключ утечёт в чат, и он потом ещё будет вам рекомендовать его ротировать, а там ещё бюрократия.»

Что делать:

1. **Права на уровне ОС.** «Самое простое — запретить конкретному файлу read-write вообще жёстко, на всю систему, только через sudo. Тогда агент не получит к нему доступ.»
2. **Плейсхолдеры.** В рабочем файле — плейсхолдер, реальный ключ подставляется отдельно.

Большой проект и локальные модели

84:50 Честная оговорка про свой опыт:

«У меня огромный плюс, что сейчас нет большого прод-проекта, который не страшно скормить. Поэтому я не могу вам сказать, как точно надо на супер-продакшн-проекте, где большая наработанная годами кодовая база, которую нельзя сливать.»

Что сказать может:

- в большой проект не запускать ИИ сразу — сначала поставить GitNexus или Graphify, чтобы агент не тратил кучу токенов на обход файлов;

- дробить проект по частям и запускать в отдельных песочницах;
- хорошая подписка (enterprise, API) декларирует, что данные не используются для обучения. «Они же могут и врать — это сейчас сплошь и рядом. Что для них один-два иска на копейки, когда они обучат модель.»

86:30 Про будущее, к которому это ведёт:

«Самый классный вариант — своё железо в локальном контуре, всё под VPN. Современная китайская модель — она open source. Kimi K3, например, — 27 миллиардов, триллион параметров, супер-огромная, на консьюмерском железе не пойдёт, но работает реально хорошо. Я сам протестил: код пишет отлично, визуал отлично, агентов спавнит, всё делает. Есть GLM 5.2, 5.3 недавно вышел, MiniMax тоже отличная. Они все будут хуже Claude или ChatGPT, но на 2, 3, 5, 10, максимум 15 %. А что такое 15 % хуже последнего Opus 5 — это тот же Opus 4.5, 4.7, который был 2–3 месяца назад, а это реально крутой уровень.»

88:00 «Вам переживать за прод-проект не стоит: когда железо закупят и запустят, вы просто открываете тот же Claude Code, в настройках прописываете, что используете не подписку Anthropic, а локальную модель через вашу сеть. И у вас безлимитные токены. Возможно, он будет не такой умный — сделайте чуть больше обвязки, и всё будет круто работать.»

Финал

88:30 Вопрос про скупку и оцифровку книг для обучения ИИ:

«Субъективно это круто. Грустно, что всё будет сосредоточено в одних руках: если обрубят данные, люди останутся без доступа к этим знаниям. Но то, что это оцифровано, скомпоновано, — мне нравится.»

89:00 «Мне вообще нравится, что ИИ появилось и упрощает жизнь. Я в целом творческий человек, люблю делать разные штуки — что руками, что в цифровом мире. Раньше очень много времени требовалось, чтобы сделать что-то не из своего стека. А сейчас ты ограничен лишь фантазией и своей идеей.»

89:10 «Да, сейчас как раз займусь вашими оценками. Всем спасибо. На связи, до скорых встреч.»